

[Menu](#)

Database Structure

▼ Relevant source files

`admin/setup/dbcreate/addProceduresTriggers.sql``admin/setup/dbcreate/addReferentialConstraints.sql``admin/setup/dbcreate/blankDBStructure.sql``admin/setup/dbcreate/coreDefinitions.txt``admin/setup/dbupgrade/DBUpgrade_0.0.0_to_1.0.0.sql``admin/setup/dbupgrade/DBUpgrade_1.0.0_to_1.1.0.sql``admin/setup/dbupgrade/DBUpgrade_1.0.0_to_1.1.0_old.sql``admin/setup/dbupgrade/DBUpgrade_1.1.0_to_1.2.0.sql``admin/setup/dbupgrade/DBUpgrade_1.2.0_to_1.3.0.sql``hclient/widgets/search/searchBuilder.html``hclient/widgets/search/searchBuilder.js``hclient/widgets/search/searchBuilderItem.js``hclient/widgets/search/searchBuilderSort.js`

Introduction

The Heurist database structure is designed as a flexible, schema-on-demand system that supports diverse research data models through a metadata-driven approach. This document explains the core database tables, their relationships, and how they work together to provide a versatile research database platform.

Unlike traditional relational databases with fixed schemas, Heurist uses a dynamic approach where record types and their fields are defined within the database itself, allowing researchers to create custom data structures without database schema modifications.

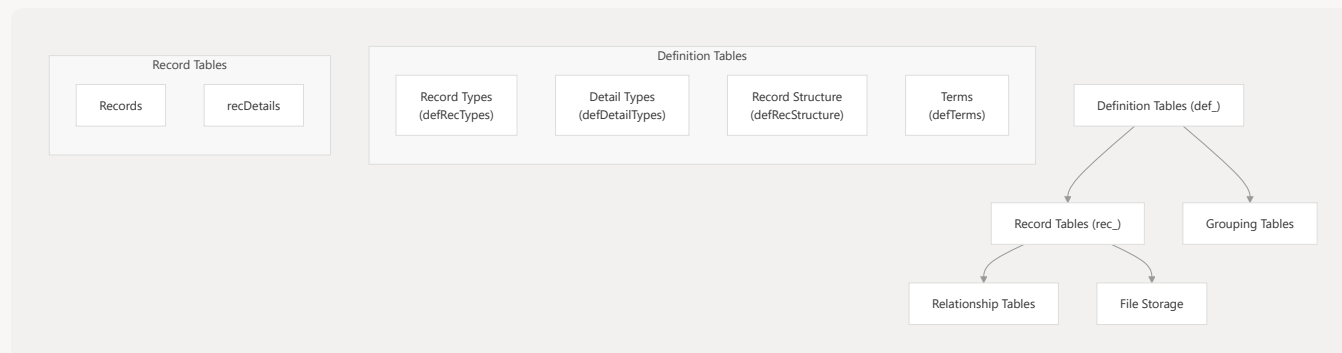
For information on configuring the database, see [Configuration System](#).

Sources:

- `admin/setup/dbcreate/blankDBStructure.sql` 15-28

Database Organization Overview

High-Level Database Structure



The Heurist database is organized into several logical sections:

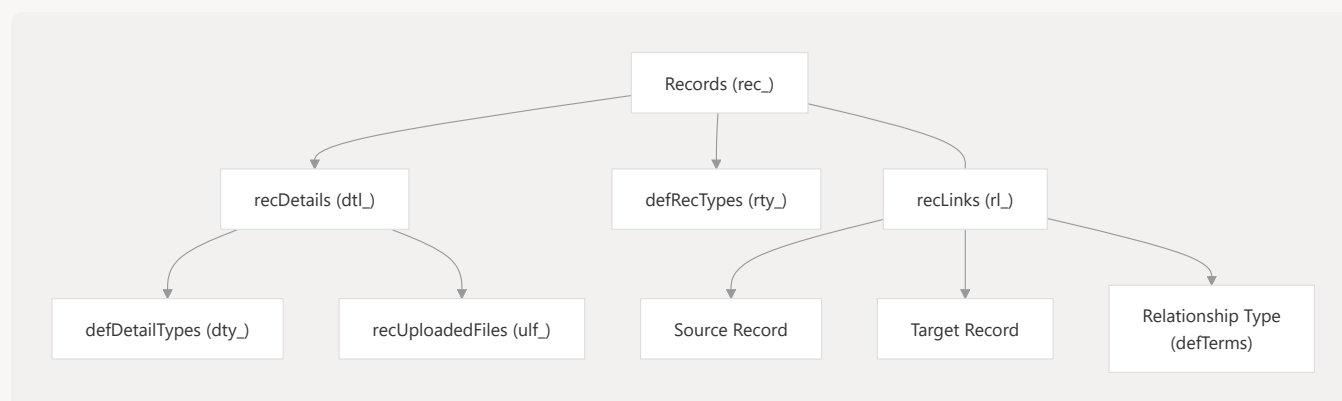
- **Definition tables** (prefixed with `def`) define the data model
- **Record tables** (prefixed with `rec`) store the actual data
- **User/system tables** (prefixed with `usr` or `sys`) store user settings and system configuration

This separation allows Heurist to function as a metadata-driven application where the data structure is itself stored as data within the database.

Sources:

- `admin/setup/dbcreate/blankDBStructure.sql` 51-82
- `admin/setup/dbcreate/blankDBStructure.sql` 137-165
- `admin/setup/dbcreate/blankDBStructure.sql` 213-255

Core Data Tables



Records Table

The `Records` table serves as the central repository for all records in the system:

Column	Type	Description
<code>rec_ID</code>	int unsigned	Primary key
<code>rec_URL</code>	varchar(2000)	Primary URL for the record
<code>rec_Title</code>	varchar(1023)	Composite title (constructed)
<code>rec_ScratchPad</code>	text	Notes field
<code>rec_RecTypeID</code>	smallint unsigned	Record type reference
<code>rec_OwnerUGrpID</code>	smallint unsigned	Owner user/group
<code>rec_NonOwnerVisibility</code>	enum	Visibility setting (viewable/hidden/public/pending)
<code>rec_Added</code>	datetime	Creation timestamp
<code>rec_Modified</code>	timestamp	Last modified timestamp
<code>rec_Hash</code>	varchar(60)	Composite hash for duplicate detection

The record title is dynamically constructed using a title mask defined in the record type, which can incorporate values from various fields.

recDetails Table

The `recDetails` table stores the actual field values for records, implementing an Entity-Attribute-Value (EAV) model:

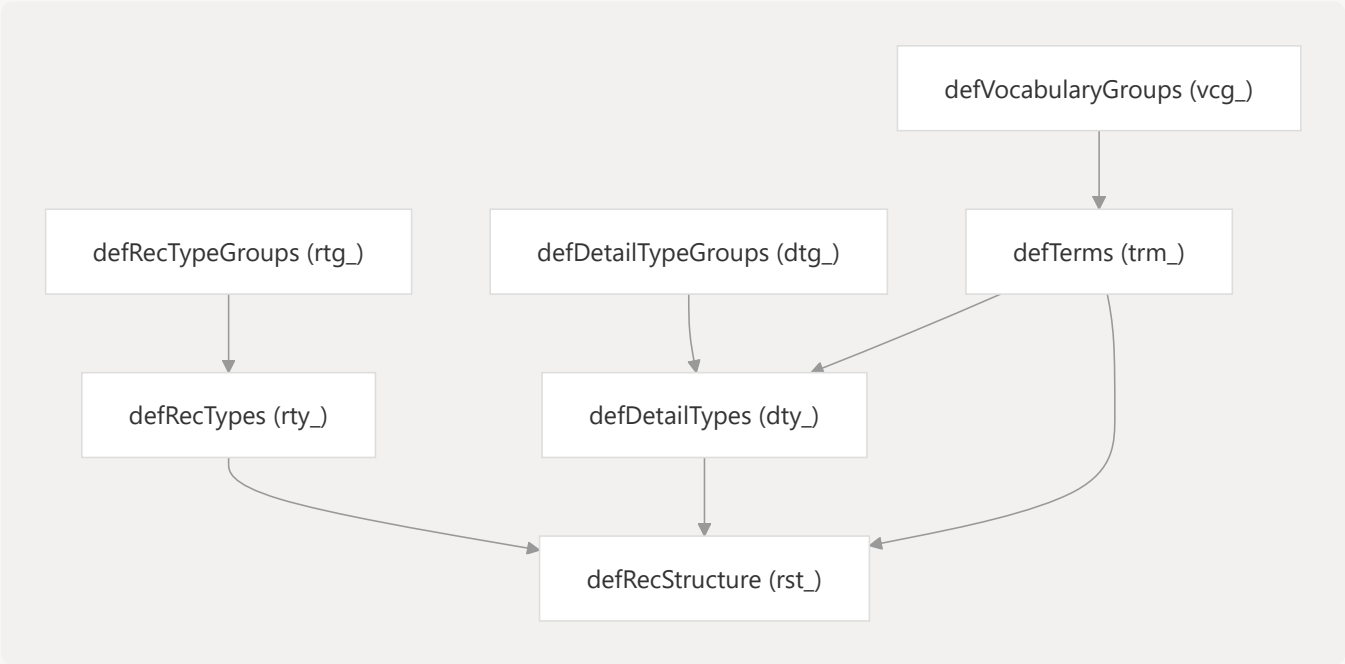
Column	Type	Description
<code>dtl_ID</code>	int unsigned	Primary key
<code>dtl_RecID</code>	int unsigned	Reference to the record
<code>dtl_DetailTypeID</code>	smallint unsigned	Field type
<code>dtl_Value</code>	text	The actual value
<code>dtl_Geo</code>	geometry	Geographical data
<code>dtl_UploadedFileID</code>	int unsigned	Reference to uploaded file
<code>dtl_Certainty</code>	decimal(3,2)	Certainty value (0-1)
<code>dtl_Annotation</code>	varchar(250)	Additional notes about the value

This EAV design enables the flexible schema where different record types can have different fields, all stored in the same table.

Sources:

- `admin/setup/dbcreate/blankDBStructure.sql` 55-82 - Records table definition
- `admin/setup/dbcreate/blankDBStructure.sql` 323-382 - recDetails references

Definition Tables



Record Type Definitions

Record types define different kinds of records (such as Person, Organization, Book, etc.):

- `defRecTypeGroups` : Groups record types into logical categories (e.g., Bibliography, People, Events)
- `defRecTypes` : Defines the record types with names, descriptions, and title masks

The record type definitions include:

Column	Type	Description
<code>rty_ID</code>	smallint unsigned	Primary key
<code>rty_Name</code>	varchar(63)	Record type name
<code>rty_Description</code>	varchar(5000)	Description
<code>rty_TitleMask</code>	varchar(500)	Formula to construct titles

Column	Type	Description
<code>rty_CanonicalTitleMask</code>	varchar(500)	Processed version of title mask
<code>rty_RecTypeGroupID</code>	tinyint unsigned	Group reference
<code>rty_OrderInGroup</code>	tinyint unsigned	Display order within group

Detail Type Definitions

Detail types define the fields that can be added to records:

- `defDetailTypeGroups` : Groups detail types into categories
- `defDetailTypes` : Defines the field types and their data types

Detail types include:

Column	Type	Description
<code>dty_ID</code>	smallint unsigned	Primary key
<code>dty_Name</code>	varchar(255)	Field name
<code>dty_Type</code>	enum	Data type (freetext, integer, date, enum, etc.)
<code>dty_DetailTypeGroupID</code>	tinyint unsigned	Group reference
<code>dty_PtrTargetRectypeIDs</code>	varchar(250)	For pointer fields, allowed target types
<code>dty_JsonTermIDTree</code>	varchar(5000)	Tree of term IDs for enumeration fields

Vocabulary Definitions

Vocabularies (controlled term lists) are defined in:

- `defVocabularyGroups` : Groups vocabularies into categories
- `defTerms` : Defines the terms used in controlled vocabularies
- `defOntologies` : References to standardized ontologies

The term definitions include:

Column	Type	Description
<code>trm_ID</code>	int unsigned	Primary key
<code>trm_Label</code>	varchar(250)	Term label
<code>trm_Description</code>	varchar(1000)	Description

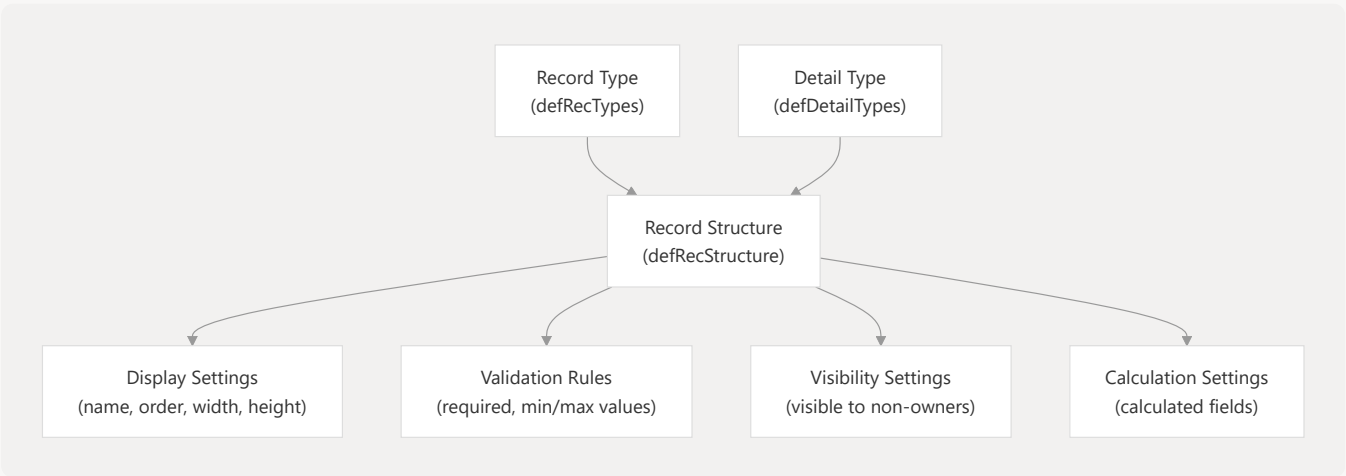
Column	Type	Description
trm_ParentTermID	int unsigned	Parent term (for hierarchical terms)
trm_Domain	enum	Domain (enum, relation)
trm_VocabularyGroupID	tinyint unsigned	Group reference
trm_Code	varchar(100)	Optional code for import/export

Sources:

- admin/setup/dbcreate/blankDBStructure.sql 263-307 - defRecTypes table definition
- admin/setup/dbcreate/blankDBStructure.sql 123-130 - defDetailTypeGroups definition
- admin/setup/dbcreate/blankDBStructure.sql 138-165 - defDetailTypes table definition
- admin/setup/dbcreate/blankDBStructure.sql 341-350 - defVocabularyGroups definition
- admin/setup/dbcreate/blankDBStructure.sql 358-382 - defTerms table definition
- admin/setup/dbcreate/coreDefinitions.txt 7-21 - Record Type Groups examples
- admin/setup/dbcreate/coreDefinitions.txt 27-48 - Detail Type Groups examples
- admin/setup/dbcreate/coreDefinitions.txt 54-69 - Vocabulary Groups examples

Record Structure

The `defRecStructure` table defines the relationship between record types and detail types, essentially specifying which fields belong to which record types and how they are displayed:



Key fields in the `defRecStructure` table include:

Column	Type	Description
rst_ID	smallint unsigned	Primary key
rst_RecTypeID	smallint unsigned	Record type reference
rst_DetailTypeID	smallint unsigned	Detail type reference
rst_DisplayName	varchar(255)	Custom display name for this field
rst_DisplayOrder	smallint unsigned	Display order in the form
rst_DisplayWidth	tinyint unsigned	Field width in the form
rst_DisplayHeight	tinyint unsigned	Field height (for text areas)
rst_RequirementType	enum	required/recommended/optional/forbidden
rst_MinValues	tinyint unsigned	Minimum values required
rst_MaxValues	tinyint unsigned	Maximum values allowed
rst_DefaultValue	text	Default value for new records
rst_CalcFunctionID	tinyint unsigned	Function for calculated fields
rst_NonOwnerVisibility	enum	Visibility outside owner group

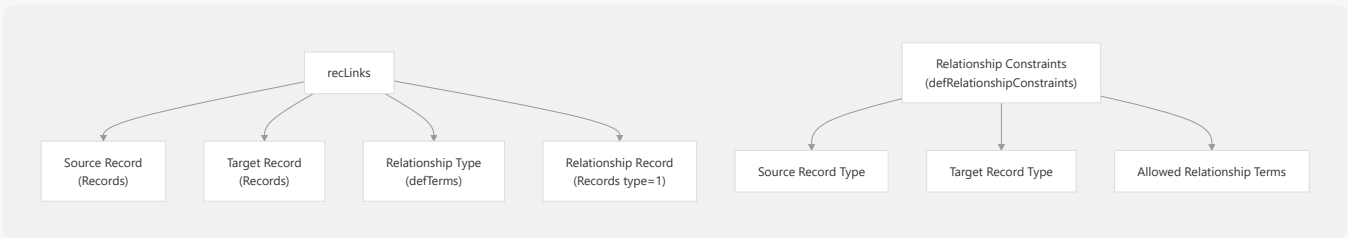
This table allows customizing how each field appears and behaves within each record type, providing significant flexibility in the data model.

Sources:

- admin/setup/dbcreate/blankDBStructure.sql 213-255 - defRecStructure table definition
- admin/setup/dbupgrade/DBUpgrade_1.1.0_to_1.2.0.sql 65-88 - Added features to defRecStructure

Relationship Management

Relationships between records are managed through:



Record Links

The `recLinks` table stores relationships between records:

Column	Type	Description
<code>rl_ID</code>	int unsigned	Primary key
<code>rl_SourceID</code>	int unsigned	Source record ID
<code>rl_TargetID</code>	int unsigned	Target record ID
<code>rl_RelationID</code>	int unsigned	ID of relationship record (for relationship records)
<code>rl_RelationTypeID</code>	int unsigned	Relationship type term ID
<code>rl_DetailTypeID</code>	smallint unsigned	Detail type (for non-relationship links)
<code>rl_DetailID</code>	int unsigned	Detail ID (for non-relationship links)

Heurist supports two types of relationships:

1. **Direct pointer fields** (a field in one record points to another record)
2. **Relationship records** (a separate record of type 1 that connects two other records)

Relationship Constraints

The `defRelationshipConstraints` table defines rules for relationships:

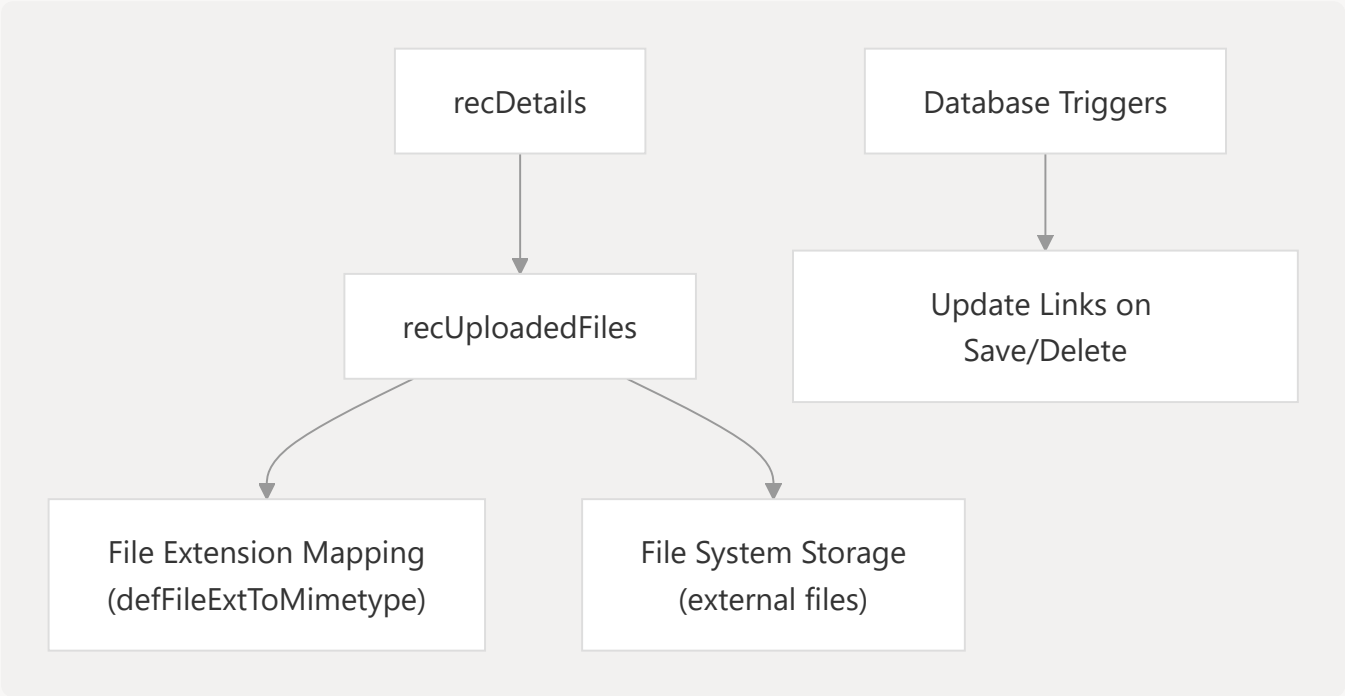
Column	Type	Description
<code>rscs_ID</code>	smallint unsigned	Primary key
<code>rscs_SourceRectypeID</code>	smallint unsigned	Source record type
<code>rscs_TargetRectypeID</code>	smallint unsigned	Target record type
<code>rscs_TermID</code>	int unsigned	Allowed relationship term
<code>rscs_TermLimit</code>	tinyint unsigned	Maximum occurrences of this term
<code>rscs_Description</code>	varchar(1000)	Description of the constraint

Sources:

- `admin/setup/dbcreate/addProceduresTriggers.sql` 284-296 - Relationship record handling
- `admin/setup/dbcreate/blankDBStructure.sql` 316-334 - `defRelationshipConstraints` table definition
- `admin/setup/dbcreate/addReferentialConstraints.sql` 89-90 - `recLinks` foreign key constraints

File Storage

Uploaded files are managed through:



Uploaded Files Management

The `recUploadedFiles` table tracks files:

Column	Type	Description
<code>ulf_ID</code>	int unsigned	Primary key
<code>ulf_OrigFileName</code>	varchar(255)	Original filename
<code>ulf_FilePath</code>	varchar(1024)	Path to the file
<code>ulf_MimeExt</code>	varchar(10)	MIME type extension
<code>ulf_ExternalFileReference</code>	varchar(1000)	URI for external files
<code>ulf_PREFERREDSource</code>	enum	'local' or 'external'

File Extension to MIME Type Mapping

The `defFileExtToMimetype` table maps file extensions to MIME types, icon files, and behavior settings:

Column	Type	Description
<code>fxm_Extension</code>	varchar(10)	File extension

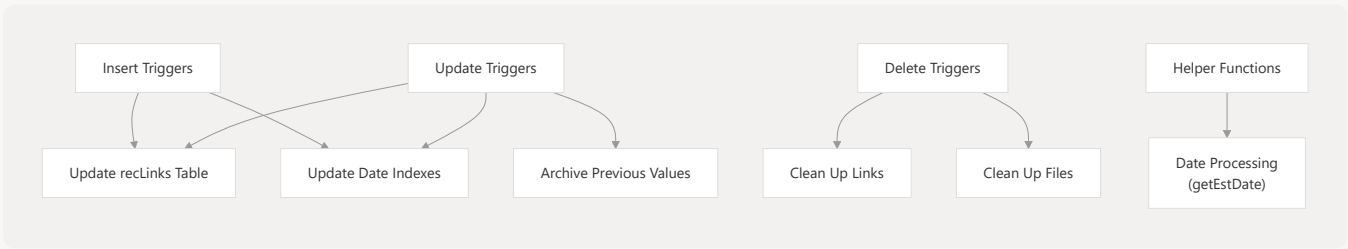
Column	Type	Description
fxm_MimeType	varchar(100)	MIME type
fxm_OpenNewWindow	tinyint unsigned	Whether to open in new window
fxm_IconFileName	varchar(31)	Icon for this file type

Sources:

- admin/setup/dbcreate/blankDBStructure.sql 173-182 - defFileExtToMimetype definition
- admin/setup/dbupgrade/DBUpgrade_1.0.0_to_1.1.0.sql 124-128 - External file references

Database Triggers and Procedures

Heurist uses triggers and stored procedures to maintain data integrity and implement complex behaviors:



Key Triggers

- `insert_record_trigger` : Sets up relationship records
- `insert_Details_trigger` : Updates record links for relationships and creates date indexes
- `update_Details_trigger` : Updates links when details are changed
- `delete_detail_trigger` : Removes associated links and file references

Date Indexing

The system maintains a `recDetailsDateIndex` table for efficient date searches, updated by triggers that extract and standardize date values.

Helper Functions

The database includes helper functions:

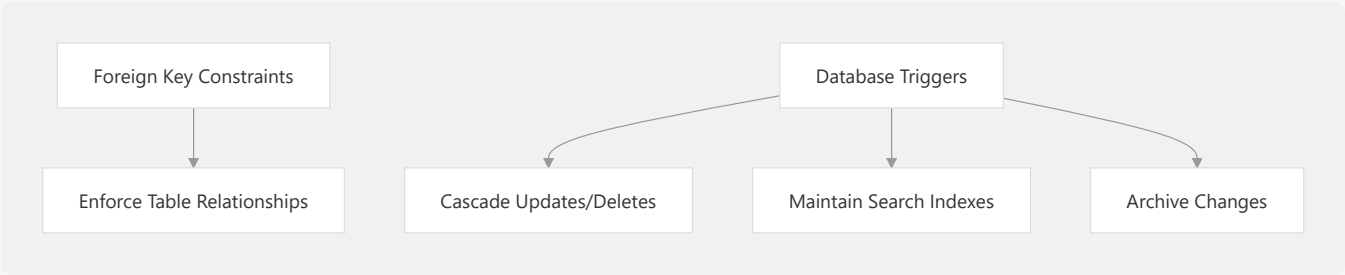
- `getEstDate` : Extracts standardized dates from various temporal formats
- `getTemporalDateString` : Processes temporal date strings

Sources:

- `admin/setup/dbcreate/addProceduresTriggers.sql` 60-119 - Date functions
- `admin/setup/dbcreate/addProceduresTriggers.sql` 272-308 - Detail insert trigger
- `admin/setup/dbcreate/addProceduresTriggers.sql` 344-382 - Detail update trigger
- `admin/setup/dbcreate/addProceduresTriggers.sql` 392-406 - Detail delete trigger
- `admin/setup/dbcreate/addProceduresTriggers.sql` 412-427 - Record insert trigger

Data Integrity

The database uses foreign key constraints and triggers to maintain data integrity:



Foreign key constraints enforce relationships between tables:

Constraint	From Table	From Field	To Table	To Field
fk_rec_RecTypeID	Records	rec_RecTypeID	defRecTypes	rty_ID
fk_rec_OwnerUGrpID	Records	rec_OwnerUGrpID	sysUGrps	ugr_ID
fk_dtl_RecID	recDetails	dtl_RecID	Records	rec_ID
fk_dtl_DetailTypeID	recDetails	dtl_DetailTypeID	defDetailTypes	dty_ID
fk_rst_RecTypeID	defRecStructure	rst_RecTypeID	defRecTypes	rty_ID
fk_rst_DetailTypeID	defRecStructure	rst_DetailTypeID	defDetailTypes	dty_ID

Triggers maintain data integrity across tables automatically, ensuring that:

- Changes to records are properly archived
- Relationship links are kept in sync
- Date indexes are maintained for efficient searching

Sources:

- `admin/setup/dbcreate/addReferentialConstraints.sql` 35-39 - Records table constraints
- `admin/setup/dbcreate/addReferentialConstraints.sql` 67-69 - RecStructure constraints
- `admin/setup/dbcreate/addReferentialConstraints.sql` 98-101 - recDetails constraints

Database Schema Evolution

The database schema has evolved over time, with major versions tracked in the upgrade scripts:

Version	Major Changes
1.0.0	Base structure
1.1.0	Added visibility settings, modification timestamps
1.2.0	Added certainty, annotations, child records
1.3.0	Added vocabulary groups, permissions, working subsets

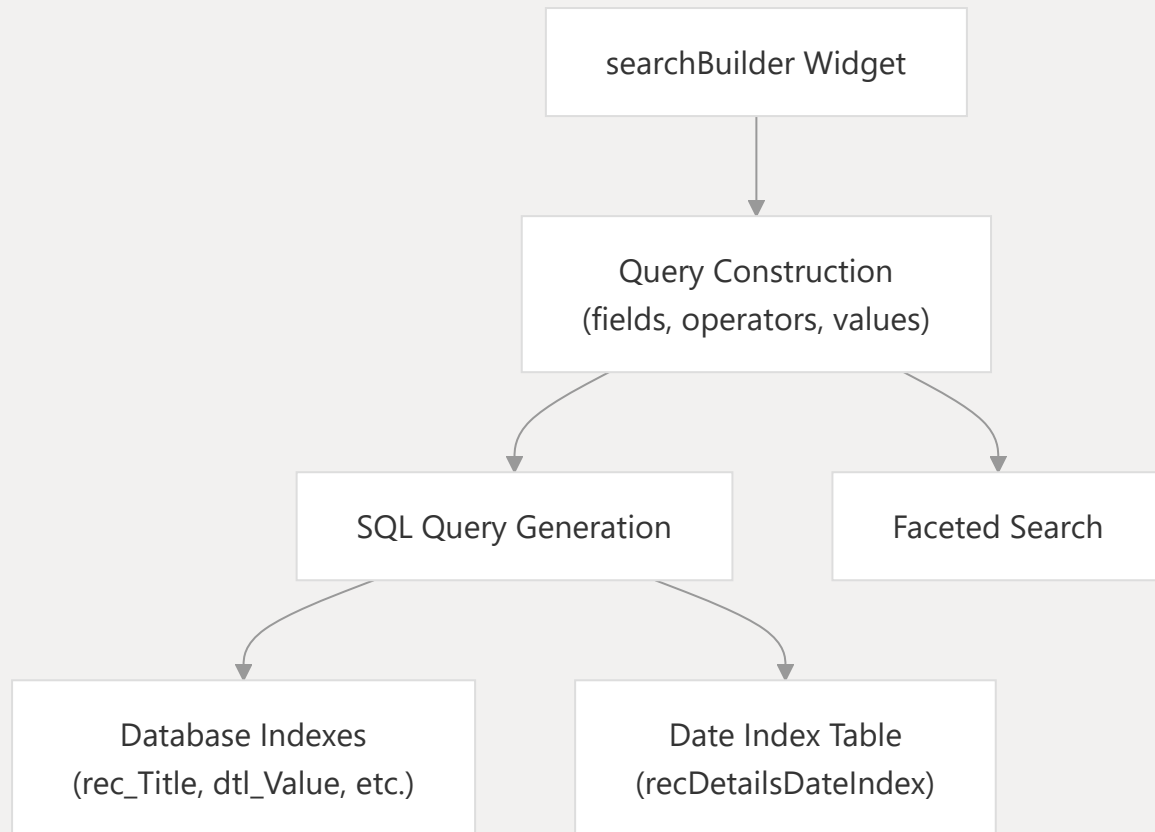
Each upgrade script maintains backward compatibility while adding new capabilities to the database structure.

Sources:

- `admin/setup/dbupgrade/DBUpgrade_0.0.0_to_1.0.0.sql` 33-45
- `admin/setup/dbupgrade/DBUpgrade_1.0.0_to_1.1.0.sql` 40-43
- `admin/setup/dbupgrade/DBUpgrade_1.1.0_to_1.2.0.sql` 31-36
- `admin/setup/dbupgrade/DBUpgrade_1.2.0_to_1.3.0.sql` 40-43

Search Implementation

The database structure supports efficient searching through:



Heurist implements several specialized indexes:

- Full-text indexes on titles and text fields
- Special date index table for temporal searches
- Spatial indexes for geographical data

The search system can filter records based on:

- Record types
- Field values
- Relationships to other records
- Geographical location
- Temporal data
- Record metadata (creation date, owner, etc.)

Sources:

- `hclient/widgets/search/searchBuilder.js` 25-143 - Search builder implementation
- `hclient/widgets/search/searchBuilderItem.js` 21-200 - Search field item implementation
- `hclient/widgets/search/searchBuilderFacet.js` 21-200 - Search facet implementation

